PES UNIVERISTY

OPERATING SYSTEM BANANA PROBLEM ASSIGNMENT

NAME:PAVITHRA N

SRN:PES2UG24CS347

1A

```
Pavithra@Pavithra-lenovo:~$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b
9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe
9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
Pavithra@Pavithra-lenovo:~$
```

1B

```
Pavithra@Pavithra-lenovo:~$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
Pavithra@Pavithra-lenovo:~$
```

2A

```
Pavithra@Pavithra-lenovo:~$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization
Stored tree object hash: 26198bf54d24ffda25c593cf008d4e99171cbe647eff872540c3b8f
4dd3a0446
Object stored at: .pes/objects/26/198bf54d24ffda25c593cf008d4e99171cbe647eff8725
40c3b8f4dd3a0446
PASS: tree object store roundtrip

All Phase 2 tests passed.
Pavithra@Pavithra-lenovo:~$
```

2B

```
Pavithra@Pavithra-lenovo:~$ find .pes/objects -type f
.pes/objects/26/198bf54d24ffda25c593cf008d4e99171cbe647eff872540c3b8f4dd3a0446
Pavithra@Pavithra-lenovo:~$ xxd .pes/objects/26/198bf54d24ffda25c593cf008d4e9917
1cbe647eff872540c3b8f4dd3a0446
00000000: 7472 6565 2039 3000 3130 3036 3434 2061  tree 90.100644 a
00000010: 2e74 7874 00dd dddd dddd dddd dddd dddd  .txt............
00000020: dddd dddd dddd dddd dddd dddd dddd dddd  ................
00000030: dddd dddd dd31 3030 3634 3420 622e 7478  .....100644 b.tx
00000040: 7400 eeee eeee eeee eeee eeee eeee eeee  t...............
00000050: eeee eeee eeee eeee eeee eeee eeee eeee  ................
00000060: eeee                                     ..
Pavithra@Pavithra-lenovo:~$
```

3A

```
Pavithra@Pavithra-lenovo:~$ make pes
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index    # Human-readable text format
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  pes.h
  untracked:  test_objects.c
  untracked:  index.c
  untracked:  commit.c
  untracked:  test_objects
  untracked:  tree.h
  untracked:  test_tree
  untracked:  Makefile
  untracked:  index.h
  untracked:  commit.h
  untracked:  test_sequence.sh
  untracked:  tree.c
  untracked:  pes.c
  untracked:  .gitignore
  untracked:  test_tree.c
  untracked:  README.md
  untracked:  object.c

100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776680907 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776680907 6 file2.txt
Pavithra@Pavithra-lenovo:~$
```

3B

```
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776680907 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776680907 6 file2.txt
Pavithra@Pavithra-lenovo:~$
```

4A

```
Pavithra@Pavithra-lenovo:~$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"

./pes log
Initialized empty PES repository in .pes/
Committed: a42f5efdd5e4... Initial commit
Committed: b140d6bd5235... Add world
Committed: 3e3982b1f342... Add farewell
commit 3e3982b1f3420f5d35627b9db2f229c7cb111932e6b73f4ca03245bb49ba4690
Author: Pavithra <PES2UG24CS347>
Date:   1776680985

    Add farewell

commit b140d6bd52352a6c7bfd184ee60214318e45e73f276e397999e9d5c5f0eb4b56
Author: Pavithra <PES2UG24CS347>
Date:   1776680985

    Add world

commit a42f5efdd5e4776db65c8c6e62eae1d65f35894c5fc6b6062a3cabd2f5e9d8f4
Author: Pavithra <PES2UG24CS347>
Date:   1776680985

    Initial commit

Pavithra@Pavithra-lenovo:~$
```

4B

```
Pavithra@Pavithra-lenovo:~$ find .pes/objects -type f | sort
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/26/198bf54d24ffda25c593cf008d4e99171cbe647eff872540c3b8f4dd3a0446
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/3e/3982b1f3420f5d35627b9db2f229c7cb111932e6b73f4ca03245bb49ba4690
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/a4/2f5efdd5e4776db65c8c6e62eae1d65f35894c5fc6b6062a3cabd2f5e9d8f4
.pes/objects/b1/40d6bd52352a6c7bfd184ee60214318e45e73f276e397999e9d5c5f0eb4b56
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
Pavithra@Pavithra-lenovo:~$
```

4C

```
Pavithra@Pavithra-lenovo:~$ cat .pes/refs/heads/main
3e3982b1f3420f5d35627b9db2f229c7cb111932e6b73f4ca03245bb49ba4690
Pavithra@Pavithra-lenovo:~$ cat .pes/HEAD
ref: refs/heads/main
Pavithra@Pavithra-lenovo:~$
```

Final (Integration Testing)

```
Pavithra@Pavithra-lenovo:~$ bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: f96a813023a9... Initial commit

Log after first commit:
commit f96a813023a95253c49f23924fcacb283a3cc39fb328d64e8710d4e3aa10a9e8
Author: Pavithra <PES2UG24CS347>
Date:   1776681727

    Initial commit


--- Second Commit ---
Committed: 922ed34e3d89... Update file.txt

--- Third Commit ---
Committed: 3674df7e763d... Add farewell

--- Full History ---
commit 3674df7e763d70cf4d99dc2644cee150bdf9b9edf62b451a04c1fc335d3ca54c
Author: Pavithra <PES2UG24CS347>
Date:   1776681727
```

```
Committed: 3674df7e763d... Add farewell

--- Full History ---
commit 3674df7e763d70cf4d99dc2644cee150bdf9b9edf62b451a04c1fc335d3ca54c
Author: Pavithra <PES2UG24CS347>
Date:   1776681727

    Add farewell

commit 922ed34e3d89f9f2682a8757bc4833c3f6ea546a624335d7cbe3cec03ec620d7
Author: Pavithra <PES2UG24CS347>
Date:   1776681727

    Update file.txt

commit f96a813023a95253c49f23924fcacb283a3cc39fb328d64e8710d4e3aa10a9e8
Author: Pavithra <PES2UG24CS347>
Date:   1776681727

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
3674df7e763d70cf4d99dc2644cee150bdf9b9edf62b451a04c1fc335d3ca54c

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/36/74df7e763d70cf4d99dc2644cee150bdf9b9edf62b451a04c1fc335d3ca54c
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/92/2ed34e3d89f9f2682a8757bc4833c3f6ea546a624335d7cbe3cec03ec620d7
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f9/6a813023a95253c49f23924fcacb283a3cc39fb328d64e8710d4e3aa10a9e8

=== All integration tests completed ===
test_sequence.sh: line 82: //: Is a directory
Pavithra@Pavithra-lenovo:~$
```

Q5.1

A checkout operation updates both repository metadata and the working directory. In .pes, the HEAD file must be updated to point to the selected branch, typically in the form "ref: refs/heads/branchname". The branch file itself already contains the commit hash, so it does not change.

The system then resolves the branch to a commit, reads the commit object to obtain its tree, and reconstructs the working directory from that tree. This involves creating files that exist in the target tree, overwriting files that differ, and deleting files that are not present in the target tree. After updating the working directory, the index must also be rewritten so that it reflects the new tree state.

The operation is complex because it must avoid overwriting uncommitted changes. It requires comparing the working directory, index, and target tree. It also has to handle directory structures, file mode differences, and ensure the update is consistent even if interrupted

Q5.2

To detect a dirty working directory, compare each tracked file in the index with the current working directory version. For each entry, read the file from disk, compute its hash using the same object hashing method, and compare it with the hash stored in the index. If they differ, the file has been modified.

Next, compare the index with the target branch’s tree. If a file differs between the current index and the target tree, and also differs between the index and the working directory, then there is a conflict. In this case, checkout must be refused to prevent loss of uncommitted changes.

This approach uses only the index and object store, without needing extra metadata

Q5.3

In a detached HEAD state, HEAD directly stores a commit hash instead of pointing to a branch. If new commits are created in this state, they are not referenced by any branch. They still exist in the object store, but no named reference points to them.

These commits can be recovered if the user still knows their hash or if they are visible through mechanisms like a reflog. The user can create a new branch pointing to that commit, making it reachable again. If no reference is created, the commits may eventually be removed by garbage collection

Q6.1

To find unreachable objects, start from all branch references in refs/heads and mark their commit hashes as reachable. Traverse each commit by following its parent links and associated tree objects. For each tree, recursively visit all subtree objects and blobs. Mark every visited object as reachable.

After traversal, iterate through all objects in the object store and delete those not marked as reachable.

A hash set is an efficient data structure for tracking reachable objects, as it allows constant-time insertion and lookup.

For a repository with 100000 commits and 50 branches, the number of unique objects visited is approximately the number of commits plus associated trees and blobs. Since many commits share history, the traversal will likely visit around 100000 commits, not 50 times that number. Including trees and blobs, the total could be a few hundred thousand objects.

Q6.2

Running garbage collection concurrently with a commit operation is dangerous because of race conditions. For example, a commit operation may create new objects (blobs, trees) and compute their hashes, but before the commit reference is written, these objects are not yet reachable from any branch. If garbage collection runs at that moment, it may consider these objects unreachable and delete them.

This leads to a corrupted commit that references missing objects.

Git avoids this by ensuring that new objects are safely written and referenced before they can be collected. It uses mechanisms such as temporary files, atomic renaming, and object protection during operations. Additionally, Git's garbage collection is typically not run concurrently with active writes, or it uses safeguards like packfiles and reachability checks that prevent recently created objects from being deleted prematurely